Exam Preparation & Practice

Kubernetes Production · Lesson 23

Strategy, speed, and practice scenarios for exam day

Learning Objectives

By the end of this lesson you will be able to:

- Describe the exam logistics and rules
- Prioritise study time by domain weight
- Work fast with imperative `kubectl` and `--dry-run`
- Apply a repeatable per-task and time-management strategy
- Run two end-to-end practice scenarios

> Practice on a real cluster until these patterns are automatic.

23.1 Exam Logistics

Exam Day Logistics

Item	Detail
Format	Performance-based tasks in a browser terminal
Duration	**2 hours**
Passing	**66%**
Version	**Kubernetes v1.35**
Proctoring	Remote, ID required, clean desk
Allowed docs	`kubernetes.io/docs` (+ subdomains), nothing else
Retake	One free retake within 12 months

> You get **multiple clusters** — each task tells you which `kubectl` **context** to use. Switching context first, every time, prevents costly mistakes.

Always Switch Context First

Each question specifies a cluster. Begin **every** task with:

```
kubectl config use-context <context-from-the-question>
```

And work in the right namespace:

```
kubectl config set-context --current --namespace=<ns>
```

> A correct answer on the **wrong cluster** scores **zero**. Make context-switch your reflex before reading the rest of the task.

23.2 Study Strategy

Prioritise by Domain Weight

Domain	Weight	Focus
Troubleshooting	30%	logs, events, describe, fix broken Pods/nodes
Cluster Architecture	25%	kubeadm, etcd backup/restore, RBAC, upgrades
Services & Networking	20%	Services, Ingress, NetworkPolicy, DNS
Workloads & Scheduling	15%	Deployments, taints/affinity, resources
Storage	10%	PV, PVC, StorageClass

> Spend the most practice time on **Troubleshooting** and **Architecture** — together they are **55%** of the score.

New 2025 Topics

The refreshed CKA adds real-world tooling. Be comfortable with:

- **Helm** — install/upgrade charts (`helm install`, `helm upgrade`)
- **Kustomize** — `kubectl apply -k`, overlays and patches
- **Gateway API** — the successor to Ingress (`GatewayClass`, `Gateway`, `HTTPRoute`)
- **CRDs & Operators** — `kubectl get crd`, custom resources

> All now covered in this course: **Helm & Kustomize (L12)**, **CRDs & Operators (L13)**, **Gateway API (L5)**, **autoscaling (L3)**, **HA control plane (L2)**.

23.3 Working Fast

Speed Setup

Configure your shell in the first minute:

```
alias k=kubectl
export do='--dry-run=client -o yaml'     # generate manifests fast
export now='--force --grace-period=0'    # delete immediately
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
```

> `kubectl` autocompletion + a short alias save real minutes across 15+ tasks.

Generate, Don't Hand-Write YAML

Start from generated manifests, then edit:

```
# Deployment skeleton
k create deploy web --image=nginx:1.27 --replicas=3 $do > web.yaml

# Pod skeleton
k run tmp --image=busybox $do --command -- sleep 3600 > pod.yaml

# Service / expose
k expose deploy web --port=80 --target-port=8080 $do > svc.yaml

# RBAC
k create role r --verb=get,list --resource=pods $do
k create rolebinding rb --role=r --serviceaccount=ns:sa $do
```

> Imperative-then-edit is far faster than writing YAML from scratch.

Look It Up Fast

```
kubectl explain pod.spec.containers.resources    # field schema, offline
kubectl explain pvc.spec --recursive             # full tree

kubectl api-resources            # kind ↔ shortname ↔ apiVersion
kubectl get pod -o yaml | less   # see every field of a live object
```

In the docs, search for the task name (e.g. "network policy") and **copy the example YAML** straight from the page — then adapt it.

> Bookmark-by-memory: most tasks map to one docs example you can paste & tweak.

Time Management

- **~8 minutes per task** average — but weights vary, watch the % shown.
- **Triage first**: do quick, high-value tasks; **flag** hard ones and return.
- Never get stuck — partial credit counts; move on and come back.
- **Verify** each answer (`kubectl get`, `describe`, `logs`) before leaving it.
- Keep ~10 minutes at the end to re-check flagged tasks.

> Don't polish. A working, verified answer beats a perfect one you never finish.

23.4 Task Patterns Checklist

Patterns You Must Know Cold

- Create/scale/rollback a **Deployment**; update its image
- **Expose** with ClusterIP / NodePort; create an **Ingress**
- Create **PV / PVC**, bind to a Pod; pick a **StorageClass**
- **ServiceAccount + Role + RoleBinding**; verify with `auth can-i`
- **Taints / tolerations / nodeSelector / affinity**
- **etcd snapshot save / restore**
- **drain / cordon / uncordon** a node for maintenance
- **NetworkPolicy** default-deny + selective allow
- **Troubleshoot** a failing Pod or NotReady node

Troubleshooting Reflex

```
kubectl get pod -A | grep -vE 'Running|Completed'   # what's broken?
kubectl describe pod <p>          # Events at the bottom = root cause
kubectl logs <p> [-p] [-c c]      # app errors / previous crash
kubectl get events --sort-by=.lastTimestamp
kubectl top pod / nodes           # resource pressure
```

Symptom	Likely cause
`ImagePullBackOff`	wrong image/tag or registry auth
`CrashLoopBackOff`	app exits — check `logs` / command
`Pending`	no node fits (resources, taint, selector, PVC)
`OOMKilled`	memory limit too low

23.5 Practice Scenarios

Practice Exam #1 (excerpt)

In the provided cluster/namespace:

1. Create Deployment `web` (`nginx:1.27`, 3 replicas, label `tier=frontend`); expose it ClusterIP :80.
2. Add a **NetworkPolicy** allowing ingress to `web` only from Pods labelled `role=tester`.
3. Create a **200Mi PVC**, mount it in a Pod, prove data persists across Pod restarts.
4. Schedule a Pod onto a `tier=gold` node that **tolerates** a `batch` taint.
5. Fix a broken Deployment so it becomes `Available`.

> Full solution + verification: `_labs/Lesson_12_13_Practice_Exam.md`.

Practice Exam #2 (excerpt)

1. Build an `auditor` **ServiceAccount** that can `get/list/watch` in two namespaces but **nothing** in `default`; verify with `auth can-i`.
2. Take an **etcd snapshot** and confirm it with `snapshot status`.
3. **Cordon + drain** a worker for maintenance; confirm Pods moved; uncordon.
4. Deploy an app and route `Host: echo.local` to it through an **Ingress**.

> Full solution + verification: `_labs/Lesson_12_13_Practice_Exam.md`.

A Worked Example: etcd Backup

A frequent, high-value task — practise until automatic:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/snapshot.db

etcdctl snapshot status /opt/snapshot.db -w table   # verify
```

> The cert paths are standard on kubeadm clusters — memorise them.

Final Checklist

The day before

- Re-run both practice exams from memory, timed.
- Re-do any task you hesitated on.

On exam day

- Set aliases + completion in minute one.
- Switch **context** before every task; set the namespace.
- Triage, flag, verify, move on.
- Keep the docs open and search by task name.

> Confidence comes from reps. If you can do the labs blind, you are ready.

Key Takeaways

- **2 hours, 66%, v1.35**, performance-based, open `kubernetes.io/docs`
- **Switch context first** on every task — wrong cluster = zero
- Generate YAML with `--dry-run=client -o yaml`, then edit
- **Triage and verify**; never get stuck on one task
- Weight your prep toward **Troubleshooting + Architecture (55%)**

Good Luck!

You've got this.

Practice the labs · Trust your reflexes · Verify every answer